

Health Data Modalities II: Modeling over EHR & Claims Data

BINF 4002 — Machine Learning for Health · Lecture 18

Matthew McDermott

March 31, 2026

Department of Biomedical Informatics
Columbia University

Opening

Last Time: We Diagnosed the Problem

Lecture 17 established that EHR & claims data are not ordinary ML inputs:

- Generated as a **byproduct** of clinical care and billing — not designed for analysis

This Lecture

Last lecture diagnosed the pathology. This lecture asks: *what does that pathology do to modeling?*

Last Time: We Diagnosed the Problem

Lecture 17 established that EHR & claims data are not ordinary ML inputs:

- Generated as a **byproduct** of clinical care and billing — not designed for analysis
- **Irregularly** timed events, not fixed feature sets or regular grids

This Lecture

Last lecture diagnosed the pathology. This lecture asks: *what does that pathology do to modeling?*

Last Time: We Diagnosed the Problem

Lecture 17 established that EHR & claims data are not ordinary ML inputs:

- Generated as a **byproduct** of clinical care and billing — not designed for analysis
- **Irregularly** timed events, not fixed feature sets or regular grids
- Observation is **informative** — sicker patients generate more data

This Lecture

Last lecture diagnosed the pathology. This lecture asks: *what does that pathology do to modeling?*

Last Time: We Diagnosed the Problem

Lecture 17 established that EHR & claims data are not ordinary ML inputs:

- Generated as a **byproduct** of clinical care and billing — not designed for analysis
- **Irregularly** timed events, not fixed feature sets or regular grids
- Observation is **informative** — sicker patients generate more data
- Absent data $\neq$ absent condition — what's missing carries meaning

This Lecture

Last lecture diagnosed the pathology. This lecture asks: *what does that pathology do to modeling?*

Last Time: We Diagnosed the Problem

Lecture 17 established that EHR & claims data are not ordinary ML inputs:

- Generated as a **byproduct** of clinical care and billing — not designed for analysis
- **Irregularly** timed events, not fixed feature sets or regular grids
- Observation is **informative** — sicker patients generate more data
- **Absent data** $\neq$ **absent condition** — what's missing carries meaning
- Selection into the dataset is **structured** (Berkson's Paradox)

This Lecture

Last lecture diagnosed the pathology. This lecture asks: *what does that pathology do to modeling?*

Today's Scope: Deliberately Simple

We will focus on **binary classification** throughout this lecture.

Why restrict ourselves?

- So you can see what changes because of the **data modality**, not because the prediction task became mathematically exotic

Survival Models

Survival analysis is often the principled generalization when time-to-event, censoring, and competing risks matter. There is a lab on this. Today: *what issues survive even if you insist on binary classification?*

Today's Scope: Deliberately Simple

We will focus on **binary classification** throughout this lecture.

Why restrict ourselves?

- So you can see what changes because of the **data modality**, not because the prediction task became mathematically exotic
- Binary classification is something you understand well from our ML fundamentals block

Survival Models

Survival analysis is often the principled generalization when time-to-event, censoring, and competing risks matter. There is a lab on this. Today: *what issues survive even if you insist on binary classification?*

Today's Scope: Deliberately Simple

We will focus on **binary classification** throughout this lecture.

Why restrict ourselves?

- So you can see what changes because of the **data modality**, not because the prediction task became mathematically exotic
- Binary classification is something you understand well from our ML fundamentals block
- Even with this “simple” task, the health context forces special handling

Survival Models

Survival analysis is often the principled generalization when time-to-event, censoring, and competing risks matter. There is a lab on this. Today: *what issues survive even if you insist on binary classification?*

Today's Plan

1. **Task definition:** actionability, censoring, competing risks ~10 min
2. **Three representations:** tabularized, chunked, event stream ~25 min
3. **What actually changes in the model** ~25 min
4. **Evaluation callout & synthesis** ~10 min

Core Question

If I keep the prediction objective simple — binary classification — what changes because the input is health event-stream data rather than ordinary ML data?

Task Definition

A Prediction Must Align with a Decision

“Predict **X**” is incomplete unless we specify:

- **For whom?** — Which patients are eligible (cohort)?

A Prediction Must Align with a Decision

“Predict **X**” is incomplete unless we specify:

- **For whom?** — Which patients are eligible (cohort)?
- **From when?** — What is the prediction time / index event?

A Prediction Must Align with a Decision

“Predict **X**” is incomplete unless we specify:

- **For whom?** — Which patients are eligible (cohort)?
- **From when?** — What is the prediction time / index event?
- **Using what?** — What history is available at that moment?

A Prediction Must Align with a Decision

“Predict X” is incomplete unless we specify:

- **For whom?** — Which patients are eligible (cohort)?
- **From when?** — What is the prediction time / index event?
- **Using what?** — What history is available at that moment?
- **Over what horizon?** — What is the prediction window?

A Prediction Must Align with a Decision

“Predict X” is incomplete unless we specify:

- **For whom?** — Which patients are eligible (cohort)?
- **From when?** — What is the prediction time / index event?
- **Using what?** — What history is available at that moment?
- **Over what horizon?** — What is the prediction window?
- **To support what action?** — What clinical decision does this inform?

Good vs. Bad Task Definitions

Vague

“Will this patient die?”

- No prediction time
- No horizon
- No actionable decision
- Everyone dies eventually

Good vs. Bad Task Definitions

Vague

“Will this patient die?”

- No prediction time
- No horizon
- No actionable decision
- Everyone dies eventually

Actionable

“At admission, using data available at that time, predict in-hospital mortality to guide ICU escalation.”

- Clear index event
- Defined feature window
- Specific horizon
- Linked to a decision

Good vs. Bad Task Definitions

Vague

“Will this patient die?”

- No prediction time
- No horizon
- No actionable decision
- Everyone dies eventually

Actionable

“At admission, using data available at that time, predict in-hospital mortality to guide ICU escalation.”

- Clear index event
- Defined feature window
- Specific horizon
- Linked to a decision

Prediction time is part of the task definition. A model that predicts *after* the intervention window is useless even with a high AUROC.

Prediction-Window Censoring

In binary classification, we pretend every patient has a clear $\oplus$ or $\ominus$ label.

In EHR/claims, many patients are not observed long enough to know:

- Patient transfers to another hospital

Key Message

Not having an observed event $\neq$ being a true negative. Some of your “negatives” are actually **censored** — you simply don’t know what happened. If you ignore this, your model trains on corrupted labels.

Prediction-Window Censoring

In binary classification, we pretend every patient has a clear $\oplus$ or $\ominus$ label.

In EHR/claims, many patients are not observed long enough to know:

- Patient transfers to another hospital
- Insurance coverage ends

Key Message

Not having an observed event $\neq$ being a true negative. Some of your “negatives” are actually **censored** — you simply don’t know what happened. If you ignore this, your model trains on corrupted labels.

Prediction-Window Censoring

In binary classification, we pretend every patient has a clear $\oplus$ or $\ominus$ label.

In EHR/claims, many patients are not observed long enough to know:

- Patient transfers to another hospital
- Insurance coverage ends
- Patient simply stops seeking care

Key Message

Not having an observed event $\neq$ being a true negative. Some of your “negatives” are actually **censored** — you simply don’t know what happened. If you ignore this, your model trains on corrupted labels.

Prediction-Window Censoring

In binary classification, we pretend every patient has a clear $\oplus$ or $\ominus$ label.

In EHR/claims, many patients are not observed long enough to know:

- Patient transfers to another hospital
- Insurance coverage ends
- Patient simply stops seeking care
- Follow-up data is incomplete or delayed

Key Message

Not having an observed event $\neq$ being a true negative. Some of your “negatives” are actually **censored** — you simply don’t know what happened. If you ignore this, your model trains on corrupted labels.

Competing Risks, Even for Binary Classification

Even if the downstream model is a binary classifier, the world contains events that make the target **impossible or change its meaning**.

Example: Predict 30-day readmission after discharge.

- A patient who dies on day 5 cannot be readmitted

Without handling this, your negative class becomes a **mixture** of: (1) true event-free patients, (2) censored patients, (3) patients for whom the event was *precluded*.

Competing risks are not merely a different loss function. They change what the binary label even means.

Competing Risks, Even for Binary Classification

Even if the downstream model is a binary classifier, the world contains events that make the target **impossible or change its meaning**.

Example: Predict 30-day readmission after discharge.

- A patient who dies on day 5 cannot be readmitted
- A patient discharged to hospice has a fundamentally different trajectory

Without handling this, your negative class becomes a **mixture** of: (1) true event-free patients, (2) censored patients, (3) patients for whom the event was *precluded*.

Competing risks are not merely a different loss function. They change what the binary label even means.

Competing Risks, Even for Binary Classification

Even if the downstream model is a binary classifier, the world contains events that make the target **impossible or change its meaning**.

Example: Predict 30-day readmission after discharge.

- A patient who dies on day 5 cannot be readmitted
- A patient discharged to hospice has a fundamentally different trajectory
- Is a death a “negative” for readmission? Different from “alive and not readmitted”

Without handling this, your negative class becomes a **mixture** of: (1) true event-free patients, (2) censored patients, (3) patients for whom the event was *precluded*.

Competing risks are not merely a different loss function. They change what the binary label even means.

Callout: The Survival Generalization

For Reference

Survival models handle these issues natively:

- Time-to-event naturally accommodates censoring
- Competing-risk frameworks distinguish event types
- The prediction target becomes $P(\text{event by time } t)$, not a flat binary label

See the survival analysis lab for hands-on practice.

For the rest of today: *what aspects of these problems remain even if we insist on binary classification?*

Callout: The Survival Generalization

For Reference

Survival models handle these issues natively:

- Time-to-event naturally accommodates censoring
- Competing-risk frameworks distinguish event types
- The prediction target becomes $P(\text{event by time } t)$, not a flat binary label

See the survival analysis lab for hands-on practice.

For the rest of today: *what aspects of these problems remain even if we insist on binary classification?*

Answer: nearly all of them. You still have to **define** the prediction time, handle patients whose observation ends early, and decide what to do with competing events.

Representation Choices

One Patient, Three Representations

Given a patient history and a binary prediction task, we can represent the input as:

1. **Tabularized summary** — fixed-length feature vector

Each is defensible. Each throws away something. Each changes what “missing” or “unobserved” means.

Key Principle

Most of the modeling differences in EHR are really **representation** differences.

One Patient, Three Representations

Given a patient history and a binary prediction task, we can represent the input as:

1. **Tabularized summary** — fixed-length feature vector
2. **Discretized temporal sequence** — regular chunks (days, weeks, visits)

Each is defensible. Each throws away something. Each changes what “missing” or “unobserved” means.

Key Principle

Most of the modeling differences in EHR are really **representation** differences.

One Patient, Three Representations

Given a patient history and a binary prediction task, we can represent the input as:

1. **Tabularized summary** — fixed-length feature vector
2. **Discretized temporal sequence** — regular chunks (days, weeks, visits)
3. **Full-resolution event stream** — raw timestamped events

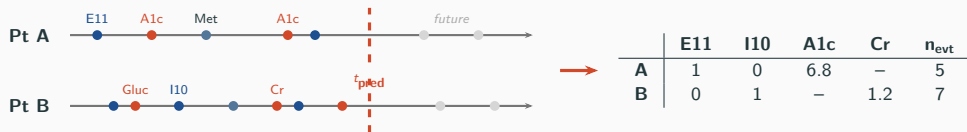
Each is defensible. Each throws away something. Each changes what “missing” or “unobserved” means.

Key Principle

Most of the modeling differences in EHR are really **representation** differences.

Representation 1: Tabularization

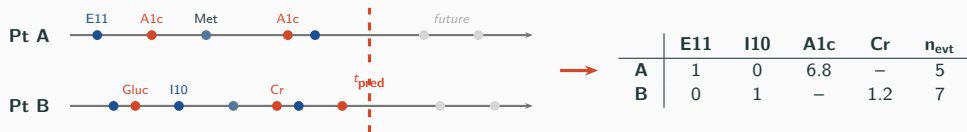
Idea: At each patient's **prediction time**, summarize the event stream *up to that point* into a fixed-length feature vector.



Critical: You only use data *up to* the prediction time. In retrospective data, future events exist in the dataset — using them is **label leakage**.

Representation 1: Tabularization

Idea: At each patient's **prediction time**, summarize the event stream *up to that point* into a fixed-length feature vector.



Critical: You only use data *up to* the prediction time. In retrospective data, future events exist in the dataset — using them is **label leakage**.

Requires choices at every step: lookback window, aggregation (counts? last value?), code granularity, recency weighting. . .

Tabularization Is a Modeling Decision

Not Just Preprocessing

Tabularization is not a neutral data preparation step. It is a **large, consequential modeling decision** that can dominate the final result.

EHR tabularized features are uniquely problematic:

- **Highly sparse:** Most patients have a tiny fraction of possible codes

Optional exercise: The ML4H MEDS Tabularization Tutorial demonstrates this.

github.com/Medical-Event-Data-Standard/MEDS_ML4H_2025_Tutorial

Tabularization Is a Modeling Decision

Not Just Preprocessing

Tabularization is not a neutral data preparation step. It is a **large, consequential modeling decision** that can dominate the final result.

EHR tabularized features are uniquely problematic:

- **Highly sparse:** Most patients have a tiny fraction of possible codes
- **Strongly dependent:** Codes co-occur through workflow, not just biology (a blood panel produces 20+ results simultaneously)

Optional exercise: The ML4H MEDS Tabularization Tutorial demonstrates this.

github.com/Medical-Event-Data-Standard/MEDS_ML4H_2025_Tutorial

Tabularization Is a Modeling Decision

Not Just Preprocessing

Tabularization is not a neutral data preparation step. It is a **large, consequential modeling decision** that can dominate the final result.

EHR tabularized features are uniquely problematic:

- **Highly sparse:** Most patients have a tiny fraction of possible codes
- **Strongly dependent:** Codes co-occur through workflow, not just biology (a blood panel produces 20+ results simultaneously)
- **Encode the care process:** Feature counts may reflect measurement behavior as much as underlying physiology

Optional exercise: The ML4H MEDS Tabularization Tutorial demonstrates this.

github.com/Medical-Event-Data-Standard/MEDS_ML4H_2025_Tutorial

Tabularization Is a Modeling Decision

Not Just Preprocessing

Tabularization is not a neutral data preparation step. It is a **large, consequential modeling decision** that can dominate the final result.

EHR tabularized features are uniquely problematic:

- **Highly sparse:** Most patients have a tiny fraction of possible codes
- **Strongly dependent:** Codes co-occur through workflow, not just biology (a blood panel produces 20+ results simultaneously)
- **Encode the care process:** Feature counts may reflect measurement behavior as much as underlying physiology
- **Ontology-sensitive:** Rolling up codes to chapter level vs. full detail changes which patterns are visible

Optional exercise: The ML4H MEDS Tabularization Tutorial demonstrates this.

github.com/Medical-Event-Data-Standard/MEDS_ML4H_2025_Tutorial

What “Unobserved” Means Under Tabularization

Recall from last lecture: *in health data, not being observed is itself informative.*

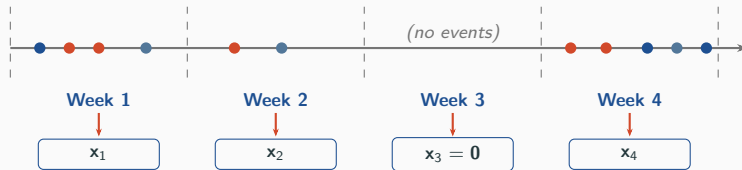
Under tabularization, this creates ambiguity in every zero or missing entry:

Feature value	Possible meanings
No diagnosis code	No disease? Not billed? Not in our system? Not captured?
No lab value	Not ordered? Normal so not flagged? Measured elsewhere?
Zero medication count	No meds? Meds from another provider? Self-pay/OTC?

Zero, missing, and truly absent are not interchangeable in health data. The representation collapses them all into the same cell.

Representation 2: Regular Chunks

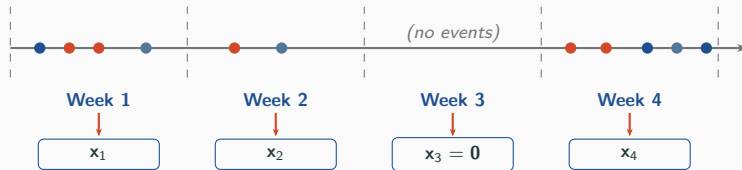
Idea: Divide the patient timeline into fixed-width bins; create a feature vector per bin.



Result: An irregularly-observed event stream becomes a *regular* sequence — compatible with RNNs, LSTMs, 1D-CNNs.

Representation 2: Regular Chunks

Idea: Divide the patient timeline into fixed-width bins; create a feature vector per bin.



Result: An irregularly-observed event stream becomes a *regular* sequence — compatible with RNNs, LSTMs, 1D-CNNs.

Note: If you fix the same number of chunks for all patients (e.g., 6 one-week bins), the result is essentially a **matrix** — a structured form of tabularization where each row is a patient and each column-group is a time window. **The binning scheme is an inductive bias.**

Chunking: Trade-offs and Observability

What it throws away:

- Within-bin ordering disappears (lab before or after medication?)

What “unobserved” means here: an empty bin is ambiguous — healthy, not seen, data from elsewhere, or no longer enrolled. Sparse vs. dense bins often encode **utilization patterns**.

The trend: Transformer-based models increasingly operate on raw event streams, reducing the *need* for chunking. But chunked representations persist for efficiency and interpretability.

Chunking: Trade-offs and Observability

What it throws away:

- Within-bin ordering disappears (lab before or after medication?)
- Simultaneous vs. same-bin events are conflated

What “unobserved” means here: an empty bin is ambiguous — healthy, not seen, data from elsewhere, or no longer enrolled. Sparse vs. dense bins often encode **utilization patterns**.

The trend: Transformer-based models increasingly operate on raw event streams, reducing the *need* for chunking. But chunked representations persist for efficiency and interpretability.

Chunking: Trade-offs and Observability

What it throws away:

- Within-bin ordering disappears (lab before or after medication?)
- Simultaneous vs. same-bin events are conflated
- The bin width imposes a clinical timescale that may not match the task

What “unobserved” means here: an empty bin is ambiguous — healthy, not seen, data from elsewhere, or no longer enrolled. Sparse vs. dense bins often encode **utilization patterns**.

The trend: Transformer-based models increasingly operate on raw event streams, reducing the *need* for chunking. But chunked representations persist for efficiency and interpretability.

Representation 3: Full-Resolution Event Stream

Idea: Keep the patient history as a sequence of timestamped events, exactly as generated.

Each event is a tuple: (time, code, value)

What this preserves:

- Fine-grained **ordering** of events

This is the **highest-fidelity** representation, used by the MEDS format and Lab 16.

Important: Code granularity is a *separate* design choice. You can operate on full ICD-10 codes, rolled-up categories, or any level. The event-stream format preserves temporal structure regardless of how you represent the codes within it.

Representation 3: Full-Resolution Event Stream

Idea: Keep the patient history as a sequence of timestamped events, exactly as generated.

Each event is a tuple: (time, code, value)

What this preserves:

- Fine-grained **ordering** of events
- Exact timing and **irregular gaps**

This is the **highest-fidelity** representation, used by the MEDS format and Lab 16.

Important: Code granularity is a *separate* design choice. You can operate on full ICD-10 codes, rolled-up categories, or any level. The event-stream format preserves temporal structure regardless of how you represent the codes within it.

Representation 3: Full-Resolution Event Stream

Idea: Keep the patient history as a sequence of timestamped events, exactly as generated.

Each event is a tuple: (time, code, value)

What this preserves:

- Fine-grained **ordering** of events
- Exact timing and **irregular gaps**
- **Repeated events** (multiple lab draws) and encounter density

This is the **highest-fidelity** representation, used by the MEDS format and Lab 16.

Important: Code granularity is a *separate* design choice. You can operate on full ICD-10 codes, rolled-up categories, or any level. The event-stream format preserves temporal structure regardless of how you represent the codes within it.

Event-Stream Modeling: New Burdens

Keeping full resolution introduces design choices that don't arise otherwise:

- **Time gaps:** How to represent 6 months of silence?

What “unobserved” means: No “no-event” tokens exist. The model must *infer* from gaps and measurement patterns what absence means. Higher-fidelity representations make care-process patterns more visible to the model, and models will exploit whatever is most predictive. We hope coarser representations filter these out — but they also discard genuine clinical signal.

Event-Stream Modeling: New Burdens

Keeping full resolution introduces design choices that don't arise otherwise:

- **Time gaps:** How to represent 6 months of silence?
- **Numeric values:** Lab result 7.2 is different from diagnosis E11.9. How to combine?

What “unobserved” means: No “no-event” tokens exist. The model must *infer* from gaps and measurement patterns what absence means. Higher-fidelity representations make care-process patterns more visible to the model, and models will exploit whatever is most predictive. We hope coarser representations filter these out — but they also discard genuine clinical signal.

Event-Stream Modeling: New Burdens

Keeping full resolution introduces design choices that don't arise otherwise:

- **Time gaps:** How to represent 6 months of silence?
- **Numeric values:** Lab result 7.2 is different from diagnosis E11.9. How to combine?
- **Simultaneous events:** An encounter can produce 50+ codes at once

What “unobserved” means: No “no-event” tokens exist. The model must *infer* from gaps and measurement patterns what absence means. Higher-fidelity representations make care-process patterns more visible to the model, and models will exploit whatever is most predictive. We hope coarser representations filter these out — but they also discard genuine clinical signal.

Event-Stream Modeling: New Burdens

Keeping full resolution introduces design choices that don't arise otherwise:

- **Time gaps:** How to represent 6 months of silence?
- **Numeric values:** Lab result 7.2 is different from diagnosis E11.9. How to combine?
- **Simultaneous events:** An encounter can produce 50+ codes at once
- **Very long sequences:** ICU patients may have >10,000 events

What “unobserved” means: No “no-event” tokens exist. The model must *infer* from gaps and measurement patterns what absence means. Higher-fidelity representations make care-process patterns more visible to the model, and models will exploit whatever is most predictive. We hope coarser representations filter these out — but they also discard genuine clinical signal.

Event-Stream Modeling: New Burdens

Keeping full resolution introduces design choices that don't arise otherwise:

- **Time gaps:** How to represent 6 months of silence?
- **Numeric values:** Lab result 7.2 is different from diagnosis E11.9. How to combine?
- **Simultaneous events:** An encounter can produce 50+ codes at once
- **Very long sequences:** ICU patients may have >10,000 events
- **Giant vocabularies:** ~70,000 ICD-10 codes plus labs, meds, procedures. . .

What “unobserved” means: No “no-event” tokens exist. The model must *infer* from gaps and measurement patterns what absence means. Higher-fidelity representations make care-process patterns more visible to the model, and models will exploit whatever is most predictive. We hope coarser representations filter these out — but they also discard genuine clinical signal.

Event-Stream Modeling: New Burdens

Keeping full resolution introduces design choices that don't arise otherwise:

- **Time gaps:** How to represent 6 months of silence?
- **Numeric values:** Lab result 7.2 is different from diagnosis E11.9. How to combine?
- **Simultaneous events:** An encounter can produce 50+ codes at once
- **Very long sequences:** ICU patients may have >10,000 events
- **Giant vocabularies:** ~70,000 ICD-10 codes plus labs, meds, procedures. . .
- **Sequence length ~ outcome:** Sicker patients generate more events — model can “cheat” by counting

What “unobserved” means: No “no-event” tokens exist. The model must *infer* from gaps and measurement patterns what absence means. Higher-fidelity representations make care-process patterns more visible to the model, and models will exploit whatever is most predictive. We hope coarser representations filter these out — but they also discard genuine clinical signal.

Comparing the Three Representations

	Tabularized	Chunked	Event Stream
Preserves	Aggregate statistics	Coarse temporal structure	Full ordering & timing
Discards	All temporal order	Within-bin order, exact timing	Nothing temporal (in principle)
Absence	Zero / missing (ambiguous)	Empty bin (visible)	Long gap (must be inferred)
Failure mode	Features encode care process	Bin width imposes wrong timescale	Model exploits observation density
Compute	Cheapest	Moderate	Most expensive
Models	Trees, logistic reg.	RNNs, 1D-CNNs	Transformers, attention

Comparing the Three Representations

	Tabularized	Chunked	Event Stream
Preserves	Aggregate statistics	Coarse temporal structure	Full ordering & timing
Discards	All temporal order	Within-bin order, exact timing	Nothing temporal (in principle)
Absence	Zero / missing (ambiguous)	Empty bin (visible)	Long gap (must be inferred)
Failure mode	Features encode care process	Bin width imposes wrong timescale	Model exploits observation density
Compute	Cheapest	Moderate	Most expensive
Models	Trees, logistic reg.	RNNs, 1D-CNNs	Transformers, attention

No representation is “correct.” The choice is a modeling decision with consequences for what your model can learn and what biases it absorbs.

What Changes in the Model

What Does Not Fundamentally Change

Before discussing what's different, what stays the same:

- You can still use **standard binary cross-entropy** loss

Key Point

Health AI is not a different discipline. The core ML machinery works. What changes is how the **input** is parameterized and what **assumptions** the data supports.

What Does Not Fundamentally Change

Before discussing what's different, what stays the same:

- You can still use **standard binary cross-entropy** loss
- You can still **optimize with SGD** and its variants

Key Point

Health AI is not a different discipline. The core ML machinery works. What changes is how the **input** is parameterized and what **assumptions** the data supports.

What Does Not Fundamentally Change

Before discussing what's different, what stays the same:

- You can still use **standard binary cross-entropy** loss
- You can still **optimize with SGD** and its variants
- **Train/val/test** splits still apply (with caveats)

Key Point

Health AI is not a different discipline. The core ML machinery works. What changes is how the **input** is parameterized and what **assumptions** the data supports.

What Does Not Fundamentally Change

Before discussing what's different, what stays the same:

- You can still use **standard binary cross-entropy** loss
- You can still **optimize with SGD** and its variants
- **Train/val/test** splits still apply (with caveats)
- **AUROC, calibration, precision-recall** still make sense

Key Point

Health AI is not a different discipline. The core ML machinery works. What changes is how the **input** is parameterized and what **assumptions** the data supports.

What Does Not Fundamentally Change

Before discussing what's different, what stays the same:

- You can still use **standard binary cross-entropy** loss
- You can still **optimize with SGD** and its variants
- **Train/val/test** splits still apply (with caveats)
- **AUROC, calibration, precision-recall** still make sense
- Regularization, hyperparameter tuning, ensembling — all still apply

Key Point

Health AI is not a different discipline. The core ML machinery works. What changes is how the **input** is parameterized and what **assumptions** the data supports.

What Changes: How to Represent Time

In text, position 1 comes before position 2 — simple. In EHR event streams, “time passing” has no single natural encoding.

Options:

- **Absolute timestamp** (Unix time or calendar date)

What Changes: How to Represent Time

In text, position 1 comes before position 2 — simple. In EHR event streams, “time passing” has no single natural encoding.

Options:

- **Absolute timestamp** (Unix time or calendar date)
- **Patient age** at event time

What Changes: How to Represent Time

In text, position 1 comes before position 2 — simple. In EHR event streams, “time passing” has no single natural encoding.

Options:

- **Absolute timestamp** (Unix time or calendar date)
- **Patient age** at event time
- **Time since previous event** (inter-event interval)

What Changes: How to Represent Time

In text, position 1 comes before position 2 — simple. In EHR event streams, “time passing” has no single natural encoding.

Options:

- **Absolute timestamp** (Unix time or calendar date)
- **Patient age** at event time
- **Time since previous event** (inter-event interval)
- **Time since last measurement of a given type**

What Changes: How to Represent Time

In text, position 1 comes before position 2 — simple. In EHR event streams, “time passing” has no single natural encoding.

Options:

- **Absolute timestamp** (Unix time or calendar date)
- **Patient age** at event time
- **Time since previous event** (inter-event interval)
- **Time since last measurement of a given type**
- **Coarse buckets** (morning/afternoon, weekday/weekend)

What Changes: How to Represent Time

In text, position 1 comes before position 2 — simple. In EHR event streams, “time passing” has no single natural encoding.

Options:

- **Absolute timestamp** (Unix time or calendar date)
- **Patient age** at event time
- **Time since previous event** (inter-event interval)
- **Time since last measurement of a given type**
- **Coarse buckets** (morning/afternoon, weekday/weekend)
- **Calendar features** (seasonality, day of week)

What Changes: How to Represent Time

In text, position 1 comes before position 2 — simple. In EHR event streams, “time passing” has no single natural encoding.

Options:

- **Absolute timestamp** (Unix time or calendar date)
- **Patient age** at event time
- **Time since previous event** (inter-event interval)
- **Time since last measurement of a given type**
- **Coarse buckets** (morning/afternoon, weekday/weekend)
- **Calendar features** (seasonality, day of week)

What Changes: How to Represent Time

In text, position 1 comes before position 2 — simple. In EHR event streams, “time passing” has no single natural encoding.

Options:

- **Absolute timestamp** (Unix time or calendar date)
- **Patient age** at event time
- **Time since previous event** (inter-event interval)
- **Time since last measurement of a given type**
- **Coarse buckets** (morning/afternoon, weekday/weekend)
- **Calendar features** (seasonality, day of week)

Each captures a different notion of “when.” Combining several is common (recall Lab 16’s learnable sinusoidal encodings).

Time Encoding: A Counter-Intuitive Result

Surprising Finding

The marginal value of sophisticated time encodings is often **lower or more task-dependent** than expected — and in some cases, even including explicit time tokens provides limited benefit over positional encoding alone.

Why might this be?

- Many signals are in the *ordering* and *co-occurrence* of events, not precise timing

This ambiguity is itself a distinctive modeling issue in health event streams — and an active research area.

Time Encoding: A Counter-Intuitive Result

Surprising Finding

The marginal value of sophisticated time encodings is often **lower or more task-dependent** than expected — and in some cases, even including explicit time tokens provides limited benefit over positional encoding alone.

Why might this be?

- Many signals are in the *ordering* and *co-occurrence* of events, not precise timing
- The observation process itself encodes rough timing through encounter density

This ambiguity is itself a distinctive modeling issue in health event streams — and an active research area.

Time Encoding: A Counter-Intuitive Result

Surprising Finding

The marginal value of sophisticated time encodings is often **lower or more task-dependent** than expected — and in some cases, even including explicit time tokens provides limited benefit over positional encoding alone.

Why might this be?

- Many signals are in the *ordering* and *co-occurrence* of events, not precise timing
- The observation process itself encodes rough timing through encounter density
- Contextual cues (seasonal diagnoses, age-related codes) provide implicit temporal signal

This ambiguity is itself a distinctive modeling issue in health event streams — and an active research area.

What Changes: Code Embeddings $\neq$ Token Embeddings

In NLP, tokens are (mostly) arbitrary symbols. Medical codes are **structured objects**:

- **Ontology hierarchy:** $E11.65 \subset E11$ (T2DM) $\subset$ Chapter IV (Endocrine)

What Changes: Code Embeddings $\neq$ Token Embeddings

In NLP, tokens are (mostly) arbitrary symbols. Medical codes are **structured objects**:

- **Ontology hierarchy:** $E11.65 \subset E11$ (T2DM) $\subset$ Chapter IV (Endocrine)
- **Text descriptions:** Every code has a human-readable name

What Changes: Code Embeddings $\neq$ Token Embeddings

In NLP, tokens are (mostly) arbitrary symbols. Medical codes are **structured objects**:

- **Ontology hierarchy:** $E11.65 \subset E11$ (T2DM) $\subset$ Chapter IV (Endocrine)
- **Text descriptions:** Every code has a human-readable name
- **Cross-vocabulary mappings:** $ICD \leftrightarrow SNOMED \leftrightarrow OMOP$

What Changes: Code Embeddings $\neq$ Token Embeddings

In NLP, tokens are (mostly) arbitrary symbols. Medical codes are **structured objects**:

- **Ontology hierarchy:** $E11.65 \subset E11$ (T2DM) $\subset$ Chapter IV (Endocrine)
- **Text descriptions:** Every code has a human-readable name
- **Cross-vocabulary mappings:** $ICD \leftrightarrow SNOMED \leftrightarrow OMOP$
- **Co-occurrence structure:** Codes that appear together in workflows

What Changes: Code Embeddings $\neq$ Token Embeddings

In NLP, tokens are (mostly) arbitrary symbols. Medical codes are **structured objects**:

- **Ontology hierarchy:** $E11.65 \subset E11$ (T2DM) $\subset$ Chapter IV (Endocrine)
- **Text descriptions:** Every code has a human-readable name
- **Cross-vocabulary mappings:** $ICD \leftrightarrow SNOMED \leftrightarrow OMOP$
- **Co-occurrence structure:** Codes that appear together in workflows

What Changes: Code Embeddings $\neq$ Token Embeddings

In NLP, tokens are (mostly) arbitrary symbols. Medical codes are **structured objects**:

- **Ontology hierarchy:** $E11.65 \subset E11$ (T2DM) $\subset$ Chapter IV (Endocrine)
- **Text descriptions:** Every code has a human-readable name
- **Cross-vocabulary mappings:** $ICD \leftrightarrow SNOMED \leftrightarrow OMOP$
- **Co-occurrence structure:** Codes that appear together in workflows

Embedding strategies that leverage this:

- **MiME** (Choi et al., NeurIPS 2018): Multilevel embeddings from visit structure
- **DescEmb / GenHPF** (Hur et al., 2022): Embed code *text descriptions* via LMs — enables cross-site transfer without shared vocabularies
- **MedTok** (2025): Multimodal tokenizer integrating text + ontology graphs

What Changes: Code Embeddings $\neq$ Token Embeddings

In NLP, tokens are (mostly) arbitrary symbols. Medical codes are **structured objects**:

- **Ontology hierarchy:** $E11.65 \subset E11$ (T2DM) $\subset$ Chapter IV (Endocrine)
- **Text descriptions:** Every code has a human-readable name
- **Cross-vocabulary mappings:** $ICD \leftrightarrow SNOMED \leftrightarrow OMOP$
- **Co-occurrence structure:** Codes that appear together in workflows

Embedding strategies that leverage this:

- **MiME** (Choi et al., NeurIPS 2018): Multilevel embeddings from visit structure
- **DescEmb / GenHPF** (Hur et al., 2022): Embed code *text descriptions* via LMs — enables cross-site transfer without shared vocabularies
- **MedTok** (2025): Multimodal tokenizer integrating text + ontology graphs

In NLP, the vocabulary is discovered from data. In health AI, it comes with pre-existing structure that can be exploited — or ignored at your peril.

EHR Foundation Models: The Autoregressive Approach

Core idea: Train a transformer to predict the next event in a patient's timeline.

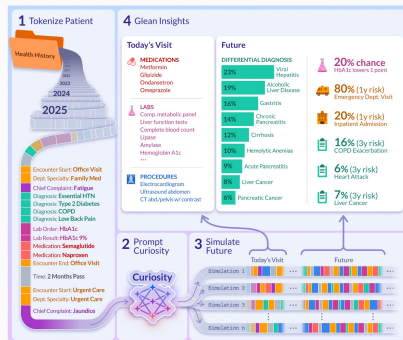


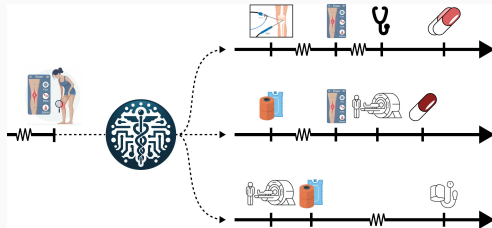
Figure: Curiosity pipeline (Epic; arXiv 2508.12104). 1) Tokenize history. 2) Feed to AR model. 3) **Simulate many futures.** 4) Aggregate into predictions.

How AR EHR Models Make Predictions

Unlike an LLM, you can't just "ask" an EHR foundation model a question.

The inference process:

1. Feed history up to t_{pred}



One patient $\rightarrow$ model $\rightarrow$ multiple sampled futures with different events and timing.

Key Point

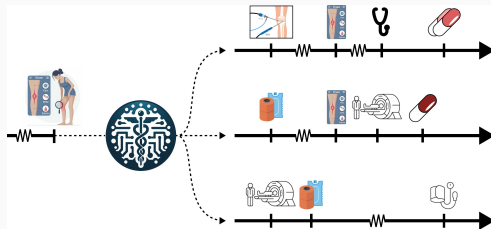
The model doesn't output a probability directly. It outputs **simulated futures**, and you derive probabilities by counting outcomes.

How AR EHR Models Make Predictions

Unlike an LLM, you can't just "ask" an EHR foundation model a question.

The inference process:

1. Feed history up to t_{pred}
2. **Autoregressively generate** many possible future trajectories



One patient $\rightarrow$ model $\rightarrow$ multiple sampled futures with different events and timing.

Key Point

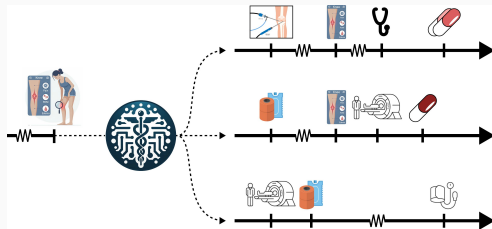
The model doesn't output a probability directly. It outputs **simulated futures**, and you derive probabilities by counting outcomes.

How AR EHR Models Make Predictions

Unlike an LLM, you can't just "ask" an EHR foundation model a question.

The inference process:

1. Feed history up to t_{pred}
2. **Autoregressively generate** many possible future trajectories
3. Each trajectory: a sequence of future events (diagnoses, labs, meds, time gaps)



One patient $\rightarrow$ model $\rightarrow$ multiple sampled futures with different events and timing.

Key Point

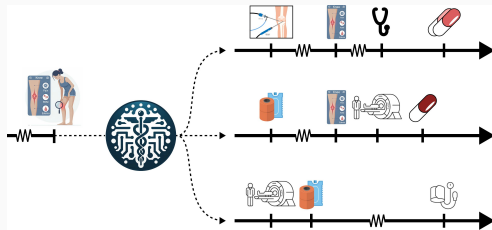
The model doesn't output a probability directly. It outputs **simulated futures**, and you derive probabilities by counting outcomes.

How AR EHR Models Make Predictions

Unlike an LLM, you can't just "ask" an EHR foundation model a question.

The inference process:

1. Feed history up to t_{pred}
2. **Autoregressively generate** many possible future trajectories
3. Each trajectory: a sequence of future events (diagnoses, labs, meds, time gaps)
4. **Aggregate** across simulations: "in what fraction did event X occur within 30 days?"



One patient $\rightarrow$ model $\rightarrow$ multiple sampled futures with different events and timing.

Key Point

The model doesn't output a probability directly. It outputs **simulated futures**, and you derive probabilities by counting outcomes.

How EHR FMs Differ from LLMs (I)

This is not just “GPT but for medical codes.” Several deep differences:

1. **No prompting.** You can't ask “will this patient develop sepsis?” You must generate full trajectories and check. With the right prompt, an LLM needs *one* forward pass for one logit. An AR EHR model needs *many* generation steps $\times$ many samples.

How EHR FMs Differ from LLMs (I)

This is not just “GPT but for medical codes.” Several deep differences:

1. **No prompting.** You can't ask “will this patient develop sepsis?” You must generate full trajectories and check. With the right prompt, an LLM needs *one* forward pass for one logit. An AR EHR model needs *many* generation steps $\times$ many samples.
2. **Calibration > MAP.** LLMs optimize for the most *likely* continuation. EHR FMs need *calibrated distributions* over futures — the 10th percentile outcome matters as much as the mode.

How EHR FMs Differ from LLMs (II)

3. **Competing generation scales.** Tokens and time are separate axes. Not all sampled trajectories will reach the prediction horizon within the token limit — this creates **systematic statistical bias** toward shorter, event-dense futures.

The Upshot

AR EHR FMs are maximally *flexible* (any question can in principle be answered from simulated futures) but pay for that flexibility with expensive, error-prone inference.

How EHR FMs Differ from LLMs (II)

3. **Competing generation scales.** Tokens and time are separate axes. Not all sampled trajectories will reach the prediction horizon within the token limit — this creates **systematic statistical bias** toward shorter, event-dense futures.
4. **Error compounding.** Because we must generate many steps to answer *any* question (no single-logit shortcut), errors compound through the autoregressive chain much more than in prompted LLMs. Every additional generated token is another chance for the model to drift.

The Upshot

AR EHR FMs are maximally *flexible* (any question can in principle be answered from simulated futures) but pay for that flexibility with expensive, error-prone inference.

EveryQuery: Adding Promptability

EveryQuery (Chandak et al.) takes a different approach: add “prompting” to EHR FMs.

The idea:

- Instead of generating full futures and aggregating, train the model to *directly answer* a specified downstream query from a single forward pass

EveryQuery: Adding Promptability

EveryQuery (Chandak et al.) takes a different approach: add “prompting” to EHR FMs.

The idea:

- Instead of generating full futures and aggregating, train the model to *directly answer* a specified downstream query from a single forward pass
- The “query” specifies: *what* to predict, *when*, over *what horizon*

EveryQuery: Adding Promptability

EveryQuery (Chandak et al.) takes a different approach: add “prompting” to EHR FMs.

The idea:

- Instead of generating full futures and aggregating, train the model to *directly answer* a specified downstream query from a single forward pass
- The “query” specifies: *what* to predict, *when*, over *what horizon*
- One pre-trained model, many tasks — without re-generating trajectories

EveryQuery: Adding Promptability

EveryQuery (Chandak et al.) takes a different approach: add “prompting” to EHR FMs.

The idea:

- Instead of generating full futures and aggregating, train the model to *directly answer* a specified downstream query from a single forward pass
- The “query” specifies: *what* to predict, *when*, over *what horizon*
- One pre-trained model, many tasks — without re-generating trajectories

EveryQuery: Adding Promptability

EveryQuery (Chandak et al.) takes a different approach: add “prompting” to EHR FMs.

The idea:

- Instead of generating full futures and aggregating, train the model to *directly answer* a specified downstream query from a single forward pass
- The “query” specifies: *what* to predict, *when*, over *what horizon*
- One pre-trained model, many tasks — without re-generating trajectories

Trade-off:

- **Gains:** Efficient inference, avoids error compounding, directly outputs calibrated probabilities
- **Costs:** Less flexible — can only answer queries the model was trained to handle

Broader tension: generative EHR FMs are maximally flexible but expensive/error-prone; query-based FMs are efficient but less general.

Callout: The Same Model on Claims Data

Everything above applies to claims — but with shifted semantics:

	EHR	Claims
Time resolution	Minutes	Days–weeks
Generating process	Bedside care	Insurance billing
Observation window	Within-system	Across providers
Incentive bias	Documentation	Upcoding / undercoding
Eligibility	Visit-based	Enrollment-based

Callout: The Same Model on Claims Data

Everything above applies to claims — but with shifted semantics:

	EHR	Claims
Time resolution	Minutes	Days–weeks
Generating process	Bedside care	Insurance billing
Observation window	Within-system	Across providers
Incentive bias	Documentation	Upcoding / undercoding
Eligibility	Visit-based	Enrollment-based

The same model may behave very differently on claims vs. EHR, because the input process means something different — even if the format looks similar.

Evaluation & Synthesis

Evaluation: Don't Forget

The representation choices from this lecture directly affect evaluation:

- Use **patient-level** splits — never same patient in train and test

Evaluation: Don't Forget

The representation choices from this lecture directly affect evaluation:

- Use **patient-level** splits — never same patient in train and test
- Consider **time-based** splits — train on earlier, test on later

Evaluation: Don't Forget

The representation choices from this lecture directly affect evaluation:

- Use **patient-level** splits — never same patient in train and test
- Consider **time-based** splits — train on earlier, test on later
- Beware **label leakage** from features recorded after the prediction time

Evaluation: Don't Forget

The representation choices from this lecture directly affect evaluation:

- Use **patient-level** splits — never same patient in train and test
- Consider **time-based** splits — train on earlier, test on later
- Beware **label leakage** from features recorded after the prediction time
- Assess **calibration** and **subgroup performance**, not just AUROC

Evaluation: Don't Forget

The representation choices from this lecture directly affect evaluation:

- Use **patient-level** splits — never same patient in train and test
- Consider **time-based** splits — train on earlier, test on later
- Beware **label leakage** from features recorded after the prediction time
- Assess **calibration** and **subgroup performance**, not just AUROC
- **External validation** across sites matters

Evaluation: Don't Forget

The representation choices from this lecture directly affect evaluation:

- Use **patient-level** splits — never same patient in train and test
- Consider **time-based** splits — train on earlier, test on later
- Beware **label leakage** from features recorded after the prediction time
- Assess **calibration** and **subgroup performance**, not just AUROC
- **External validation** across sites matters

Evaluation: Don't Forget

The representation choices from this lecture directly affect evaluation:

- Use **patient-level** splits — never same patient in train and test
- Consider **time-based** splits — train on earlier, test on later
- Beware **label leakage** from features recorded after the prediction time
- Assess **calibration** and **subgroup performance**, not just AUROC
- **External validation** across sites matters

Coming later: Lectures on fairness, continuous evaluation, and deployment.

Today's Takeaways

1. **The hardest part is often not choosing a fancy model** — it is deciding what the task and input actually are

Today's Takeaways

1. **The hardest part is often not choosing a fancy model** — it is deciding what the task and input actually are
2. **Binary classification does not save you** from censoring, competing events, or informative observation

Today's Takeaways

1. **The hardest part is often not choosing a fancy model** — it is deciding what the task and input actually are
2. **Binary classification does not save you** from censoring, competing events, or informative observation
3. **Representation is the main modeling lever** — tabularized, chunked, or event stream, each with trade-offs

Today's Takeaways

1. **The hardest part is often not choosing a fancy model** — it is deciding what the task and input actually are
2. **Binary classification does not save you** from censoring, competing events, or informative observation
3. **Representation is the main modeling lever** — tabularized, chunked, or event stream, each with trade-offs
4. **EHR foundation models are a fundamentally different paradigm** than LLMs — no prompting, error compounding, competing generation scales

Today's Takeaways

1. **The hardest part is often not choosing a fancy model** — it is deciding what the task and input actually are
2. **Binary classification does not save you** from censoring, competing events, or informative observation
3. **Representation is the main modeling lever** — tabularized, chunked, or event stream, each with trade-offs
4. **EHR foundation models are a fundamentally different paradigm** than LLMs — no prompting, error compounding, competing generation scales
5. **Modern approaches reduce the need for manual tabularization**, but do not remove the underlying clinical data problems

EHR is a useful middle case: the task stays familiar, but the data-generating process forces modeling changes.

Next lectures:

- **Clinical text** — templating, copy-paste, abbreviations, how language differs from general NLP
- **Medical imaging** — where the *architecture itself* changes dramatically (U-Nets, MIL)
- **Later:** Active learning, learning to defer, causality, fairness, continuous evaluation

Questions?

Appendix: The ACES Framework for Task Definition

ACES formalizes EHR prediction task definitions as machine-readable specifications:

- Defines cohort selection, index events, prediction windows, label criteria
- Makes task definitions reproducible and shareable
- Separates “what is the clinical question?” from “how do we extract labels?”

Key insight: If two groups can't agree on whether a patient is positive or negative, the problem is the *task definition*, not the model.

github.com/justin13601/ACES

Appendix: Phenotyping as Noisy Supervision

A diagnosis code in the EHR is a **weak label**, not ground truth.

- Billed ICD codes reflect billing incentives as much as clinical judgment
- Same condition coded differently across institutions / providers / time
- Rule-based phenotyping (e.g., eMERGE) improves specificity but adds complexity
- Training labels carry systematic noise correlated with features

Key Insight

In many EHR prediction tasks, improving **label quality** matters more than improving model architecture.

Appendix: Informative Observation — A Formal View

Standard ML: Features X observed independently of outcome Y .

EHR reality: The observation *process* depends on patient state.

- $P(\text{lab ordered} \mid \text{patient state})$ is not uniform — sicker patients get more labs
- $P(\text{event recorded} \mid \text{patient in system})$ depends on system factors
- The **number of features available** is itself a feature — and confounded

A model with access to “how many observations does this patient have?” can achieve nontrivial AUROC *even without the content of those observations*.

Implication: All models leverage the most predictive features available. Ask whether your representation exposes care-process patterns that dominate physiological signal.